salesianos

TRIANA

Proyecto Intermodular de Desarrollo de Aplicaciones Multiplataforma

Junio 2026

Versión 1.0

HOJA DE CONTROL DE CAMBIOS

Versión	Descripción	Fecha
1.0	Versión inicial del documento	01/10/2025
2.0	Versión final	05/03/2026

Índice **4**
1. Introducción **4**
2. Plan de Empresa **4**
3. Descripción detallada del sistema y listado de historias de usuario **10**
4. Modelado y diseño **11**
5. Diseño **13**
6. Implementación **14**
Backend — Spring Boot 14
Paquete model 14
Paquete enums 15
Paquete repository 15
Paquete service 16
Paquete controller 16
Paquete security 17
Aplicación móvil — Flutter 17
Paquete core 17
Paquete data/models 17
Paquete data/repositories 17
Paquete presentation/blocs 18
Paquete presentation/screens 18
Paquete config/router 18
Páginas implementadas 19
Arquitectura 20
Configuración destacada 20
Esquema de tablas 21

Índice

1. Introducción

AgroApp es una aplicación diseñada para la gestión integral de maquinaria agrícola en explotaciones del sector primario. La aplicación permite a los trabajadores consultar el estado de las máquinas disponibles, gestionar sus asignaciones y registrar incidencias, todo ello desde un dispositivo móvil.

El sistema se compone de tres capas diferenciadas: una API REST desarrollada con Spring Boot que centraliza la lógica de negocio y la persistencia de datos, una aplicación web desarrollada con Angular orientada a la administración, y una aplicación móvil desarrollada con Flutter destinada al uso diario por parte de los trabajadores en campo.

2. Plan de Empresa

Incluyendo los siguientes apartados:

JUSTIFICACIÓN: El sector agrícola español gestiona actualmente su maquinaria de forma mayoritariamente manual o mediante hojas de cálculo, lo que genera ineficiencias en la asignación de equipos, pérdidas de tiempo y desconocimiento del estado real de cada máquina. AgroApp nace para digitalizar este proceso, ofreciendo una solución sencilla y accesible desde el móvil.

¿Existe competencia? Existen soluciones ERP agrícolas pero están orientadas a grandes explotaciones con presupuestos elevados. AgroApp se diferencia por su sencillez, precio accesible y enfoque específico en la gestión de maquinaria y asignaciones de personal.

¿Qué hay de novedoso? La combinación de gestión de maquinaria + asignación de trabajadores + registro de incidencias en una única app móvil, sin necesidad de formación técnica.

NOMBRE y LOGO

Nombre: AGROAPP

He elegido porque es un nombre corto y fácil de recordar, Utiliza la palabra agricultura y aplicación. Funciona bien en español y en inglés.

El color que he elegido es el verde porque está relacionado con el campo, además de ser un color llamativo que conjuga con el amarillo de la tuerca que hace referencia a la gestión de la maquinaria.

PRODUCTO (Qué vas a ofrecer, imagen o captura de la interfaz, finalidad, necesidades que cubre)

AgroApp ofrece las siguientes funcionalidades principales:

- Consulta del estado y disponibilidad de maquinaria agrícola
- Gestión de asignaciones de máquinas a trabajadores con fechas de inicio y fin
- Registro de incidencias sobre el estado de las máquinas
- Historial de uso por trabajador
- Panel de administración web para gestión global

MODELO DE NEGOCIO

AgroApp se distribuye de forma gratuita e incluye publicidad no intrusiva dentro de la aplicación móvil. En versiones futuras se contempla un modelo freemium con funcionalidades premium para explotaciones de mayor tamaño.

CONSUMIDORES Y POSIBLES CLIENTES (Tipo de consumidores, características de los destinatarios, zona en la que viven…)

- Tipo de consumidor: Explotaciones agrícolas medianas y pequeñas, cooperativas agrarias y empresas de servicios agrícolas

- Perfil del usuario: Trabajadores del campo con conocimientos básicos de smartphone, capataces y encargados de explotación

- Zona geográfica: España, con especial foco inicial en Extremadura, comunidad con alta concentración de explotaciones agrícolas y uso intensivo de maquinaria

COMPETENCIA

Competidor	Diferencia con AgroApp
Agroptima	Orientado a gestión de cultivos, no de maquinaria
Fieldin	Solución cara, para grandes empresas
Excel/papel	Sin tiempo real, sin control de incidencias

RENTABILIDAD

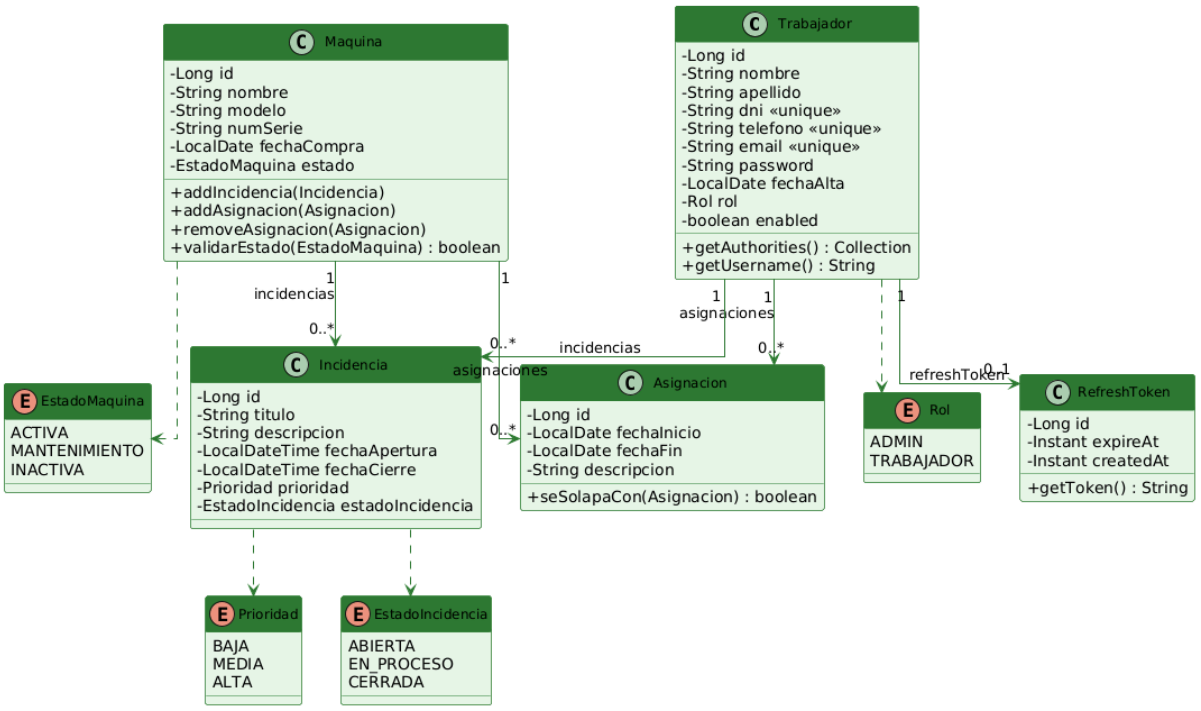
Los ingresos se generan mediante publicidad integrada en la app móvil. A futuro se contempla:

- Plan premium por suscripción mensual (~15€/mes por explotación)
- Cobro por número de trabajadores activos

3. Descripción detallada del sistema y listado de historias de usuario

ID	Rol	Historia	Criterio de aceptación
HU-01	Trabajador	Como trabajador, quiero iniciar sesión con mi email y contraseña para acceder a mis datos	Login con JWT, redirección al home
HU-02	Trabajador	Como trabajador, quiero ver mis asignaciones activas para saber qué máquina tengo asignada	Lista de asignaciones con fechas y estado
HU-03	Trabajador	Como trabajador, quiero ver el detalle de una máquina para conocer su estado y disponibilidad	Pantalla de detalle con info y asignaciones
HU-04	Trabajador	Como trabajador, quiero ver el historial de mis asignaciones pasadas para llevar un control	Lista ordenada por fecha
HU-05	Trabajador	Como trabajador, quiero registrar una incidencia sobre una máquina para notificar problemas	Formulario con descripción y máquina afectada
HU-06	Trabajador	Como trabajador, quiero crear una nueva reserva de máquina para planificar mi trabajo	Formulario con fechas y máquina seleccionada
HU-07	Admin	Como administrador, quiero gestionar las máquinas desde el panel web para mantener el inventario	CRUD completo de máquinas en Angular
HU-08	Admin	Como administrador, quiero gestionar los trabajadores para controlar el acceso al sistema	CRUD de trabajadores con roles
HU-09	Admin	Como administrador, quiero ver todas las asignaciones para tener visión global	Listado paginado con filtros
HU-10	Trabajador	Como trabajador, quiero cerrar sesión para proteger mis datos	Logout con eliminación de tokens

4. Modelado y diseño



[Enlace FIGMA]

Inicio Sesión Web

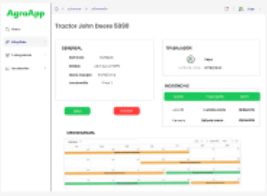
Menu Principal

Maquinas (Activas)

Maquinas (Inactivas)

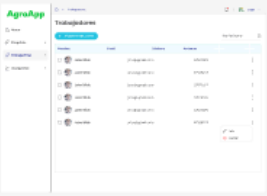
Model nueva máquina

Info Máquina

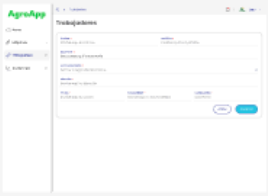


Info Maquina (EDITAR)

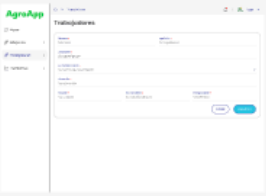
Trabajadores



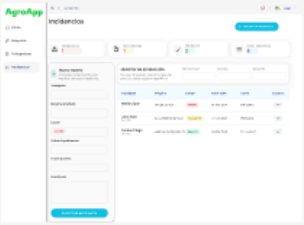
Aniadir trabajador



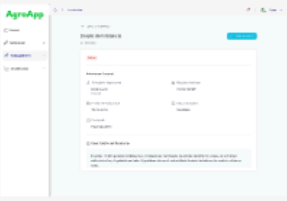
Editar



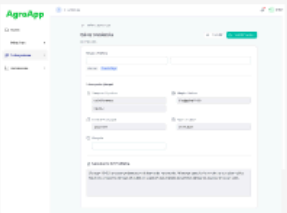
Incidencias

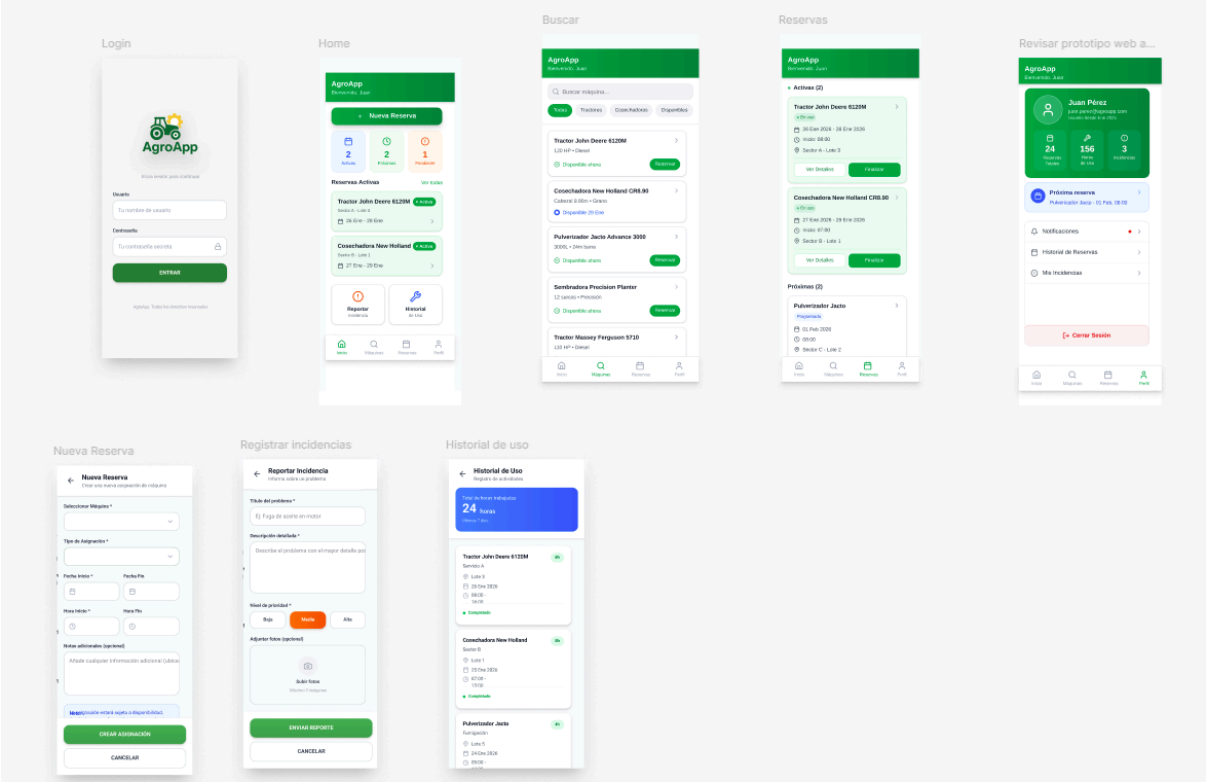


Ver incidencia

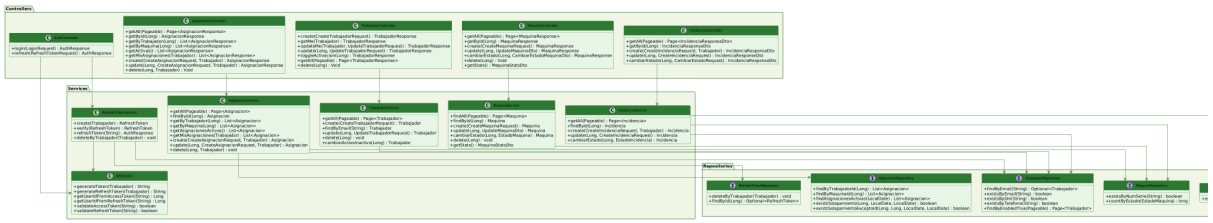


Editar Incidencia





5. Diseño



La arquitectura del backend sigue el patrón Controller → Service → Repository, con las siguientes capas:

Capa de Controladores:

- `AuthController` — gestiona el login y el refresco de tokens JWT
- `AsignacionController` — expone los endpoints REST de asignaciones con control de acceso por rol
- `MaquinaController` — CRUD de máquinas, con operaciones de cambio de estado restringidas a `ADMIN`
- `IncidenciaController` — gestión de incidencias con cambio de estado exclusivo para `ADMIN`
- `TrabajadorController` — gestión de trabajadores con operaciones de activación/desactivación

Capa de Servicios:

- `AsignacionService` — incluye lógica de validación de solapamiento de fechas y control de permisos por rol
- `MaquinaService` — valida la máquina de estados (`INACTIVA` no puede volver a `ACTIVA`) y unicidad del número de serie
- `IncidenciaService` — impide crear incidencias duplicadas sobre la misma máquina si ya hay una abierta
- `TrabajadorService` — gestiona altas, bajas lógicas y validación de unicidad de email, DNI y teléfono
- `JwtService` — genera y valida tokens de acceso y refresco con claves simétricas distintas
- `RefreshTokenService` — gestiona el ciclo de vida del refresh token persistido en base de datos

Capa de Repositorios:
Extienden `JpaRepository` de Spring Data JPA. Incluyen queries personalizadas como la detección de solapamiento de asignaciones o la búsqueda de incidencias abiertas por máquina.

6. Implementación

Backend — Spring Boot

Paquete `model`

Contiene las entidades JPA que representan el modelo de datos de la aplicación y se mapean directamente a las tablas de la base de datos.

- `Trabajador` — Entidad principal de usuario. Implementa `UserDetails` de Spring Security para integrarse con el sistema de autenticación. Gestiona roles (`ADMIN`, `TRABAJADOR`), datos personales y el estado de activación mediante el campo `enabled`. Las bajas son lógicas, nunca se elimina el registro de la base de datos.
- `Máquina` — Representa la maquinaria agrícola. Controla su propio ciclo de vida mediante el método `validarEstado()`, que impide que una máquina `INACTIVA` vuelva directamente a `ACTIVA`. Mantiene relaciones bidireccionales con `Asignacion` e `Incidencia`.
- `Asignacion` — Vincula un trabajador con una máquina durante un período de tiempo. Incluye el método `seSolapaCon()` para detectar conflictos de fechas entre asignaciones.

- `Incidencia` — Registra problemas notificados por un trabajador sobre una máquina. Tiene título, descripción, prioridad (`BAJA`, `MEDIA`, `ALTA`), estado (`ABIERTA`, `EN_PROGRESO`, `RESUELTA`) y fechas de apertura y cierre.
- `RefreshToken` — Entidad auxiliar de seguridad. Almacena el token de refresco de sesión de cada trabajador con su fecha de expiración. El token es el propio `id` del registro en base de datos.

Paquete `enums`

Define los tipos enumerados utilizados en el modelo:

- `Rol` — `ADMIN`, `TRABAJADOR`
- `EstadoMaquina` — `ACTIVA`, `MANTENIMIENTO`, `INACTIVA`
- `Prioridad` — `BAJA`, `MEDIA`, `ALTA`
- `EstadoIncidencia` — `ABIERTA`, `EN_PROGRESO`, `RESUELTA`

Paquete `repository`

Interfaces que extienden `JpaRepository` de Spring Data JPA. Proporcionan acceso a la base de datos con queries automáticas y personalizadas mediante JPQL.

- `TrabajadorRepository` — Incluye búsqueda por email, validaciones de unicidad para email, DNI y teléfono, y listado de trabajadores activos paginado.
- `MaquinaRepository` — Incluye validación de unicidad de número de serie y conteo por estado para estadísticas.
- `AsignacionRepository` — Contiene las queries más complejas del sistema: `existsSolapamiento()` y `existsSolapamientoExceptoId()` que detectan conflictos de fechas entre asignaciones, y `findAsignacionesActivas()` que filtra por fecha actual.
- `IncidenciaRepository` — Incluye `existsByMaquinaIdAndEstadoIncidenciaIn()` para impedir crear incidencias duplicadas sobre una máquina que ya tiene una abierta.
- `RefreshTokenRepository` — Proporciona búsqueda por id y eliminación por trabajador para la gestión del ciclo de vida del token.

Paquete `service`

Contiene la lógica de negocio de la aplicación. Cada servicio es transaccional y se comunica con uno o varios repositorios.

- `AsignacionService` — Gestiona el ciclo completo de asignaciones. Antes de crear o actualizar, valida que la máquina esté activa, que no haya solapamiento de fechas y que el trabajador tenga permisos. No permite eliminar asignaciones que ya han comenzado.
- `MaquinaService` — Gestiona el CRUD de máquinas. Valida la unicidad del número de serie al crear y delega la validación de cambios de estado al propio modelo (`validarEstado()`).
- `IncidenciaService` — Gestiona incidencias. Impide crear una nueva incidencia sobre una máquina que ya tiene una en estado `ABIERTA` o `EN_PROGRESO`. Al cerrar una incidencia establece automáticamente la `fechaCierre`.
- `TrabajadorService` — Gestiona el alta, modificación y baja de trabajadores. La baja es lógica (`enabled = false`). Valida unicidad de email, DNI y teléfono tanto al crear como al actualizar.
- `JwtService` — Genera y valida tokens JWT de acceso y refresco usando claves simétricas distintas configuradas en `application.properties`. El token contiene el `id` del trabajador como `subject`.
- `RefreshTokenService` — Gestiona el token de refresco persistido en base de datos. Al refrescar, genera un nuevo token de acceso y rota el refresh token (elimina el anterior y crea uno nuevo).

Paquete `controller`

Controladores REST que exponen los endpoints de la API. Delegan toda la lógica en los servicios y usan DTOs para las respuestas.

- `AuthController` — Expone `POST /api/auth/login` y `POST /api/auth/refresh`. No requiere autenticación.
- `AsignacionController` — Expone el CRUD de asignaciones en `/api/asignaciones`. Algunos endpoints están restringidos a `ADMIN` mediante `@PreAuthorize`. El endpoint `/mis-asignaciones` obtiene el trabajador autenticado mediante `@AuthenticationPrincipal`.
- `MaquinaController` — Expone el CRUD de máquinas en `/api/maquinas`. Las operaciones de escritura requieren rol `ADMIN`. Incluye endpoint `/stats` para estadísticas.
- `IncidenciaController` — Expone el CRUD de incidencias en `/api/incidencias`. El cambio de estado (`PATCH /{id}/estado`) está restringido a `ADMIN`.

- `TrabajadorController` — Expone la gestión de trabajadores en `/api/trabajadores`. Incluye endpoints `/me` para que el propio trabajador consulte y actualice sus datos sin necesidad de ser `ADMIN`.

Paquete `security`

Contiene la configuración de seguridad de la aplicación basada en Spring Security y JWT.

- `SecurityConfig` — Configura la cadena de filtros HTTP: deshabilita CSRF, establece política de sesiones `STATELESS`, define las rutas públicas (`/api/auth/**`) y protege el resto. Configura CORS para permitir peticiones desde Angular y Flutter.
- `JwtAuthenticationFilter` — Filtro que intercepta cada petición HTTP, extrae el token JWT del header `Authorization`, lo valida y establece el contexto de seguridad de Spring.
- `JwtAuthenticationEntryPoint` — Manejador que devuelve `401 Unauthorized` cuando una petición no autenticada intenta acceder a un recurso protegido.
- `JwtAccessDeniedHandler` — Manejador que devuelve `403 Forbidden` cuando un usuario autenticado intenta acceder a un recurso para el que no tiene permisos.

Aplicación móvil — Flutter

Paquete `core`

- `ApiClient` — Cliente HTTP centralizado que inyecta el token JWT en todas las peticiones. Gestiona automáticamente los errores `401` intentando refrescar el token antes de reintentar la llamada.
- `TokenManager` — Gestiona el almacenamiento seguro de tokens en `SharedPreferences`.

Paquete `data/models`

Modelos Dart que mapean las respuestas JSON del backend mediante `fromJson()` y `toJson()`. Incluyen helpers de formato para fechas (`fechaInicioFormatted`) y estado (`estadoFormatted`, `isActiva`).

Paquete `data/repositories`

Clases que encapsulan las llamadas a la API y devuelven modelos tipados a los BLoCs.

Paquete `presentation/blocs`

Implementa el patrón BLoC (Business Logic Component) para gestionar el estado de la UI de forma reactiva. Cada funcionalidad tiene su propio BLoC con eventos y estados definidos:

- `AuthBloc` — Gestiona login, logout y verificación de sesión activa
- `AsignacionBloc` — Gestiona la carga y creación de asignaciones
- `MaquinaBloc` — Gestiona el listado y detalle de máquinas
- `MaquinaAsignacionesBloc` — Gestiona las asignaciones de una máquina concreta en su pantalla de detalle
- `UsageHistoryBloc` — Gestiona el historial de asignaciones del trabajador
- `IncidenciasBloc` — Gestiona el registro de incidencias

Paquete `presentation/screens`

Pantallas de la aplicación. Cada pantalla se suscribe a su BLoC correspondiente mediante `BlocBuilder` y reacciona a los cambios de estado.

Paquete `config/router`

Configuración de navegación con GoRouter. Define las rutas de la aplicación y redirige al login si el usuario no está autenticado. Cada pantalla que necesita su propio BLoC lo recibe mediante `BlocProvider` en la propia ruta.

La aplicación Angular utiliza el sistema de rutas propio del framework, organizadas en dos niveles:

- Ruta pública: `/login` — accesible sin autenticación
- Rutas protegidas: agrupadas bajo un `AdminLayoutComponent` que actúa como layout principal con menú lateral y cabecera comunes a todas las pantallas de administración

Páginas implementadas

Ruta	Componente	Descripción

Ruta	Componente	Descripción
`/dashboard`	`DashboardComponent`	Panel principal con resumen del sistema
`/maquinas`	`MaquinasComponent`	Listado de máquinas con paginación
`/maquinas/:id`	`MaquinaDetalleComponent`	Detalle y edición de una máquina
`/trabajadores`	`TrabajadoresComponent`	Listado de trabajadores
`/trabajadores/nuevo`	`TrabajadorNuevoComponent`	Formulario de alta de trabajador
`/trabajadores/:id`	`TrabajadorDetalleComponent`	Detalle de un trabajador
`/trabajadores/:id/editar`	`TrabajadorEditarComponent`	Formulario de edición de trabajador
`/incidencias`	`IncidenciasComponent`	Listado de incidencias

`/incidencias/:id`	`IncidenciaDetalleComponent`	Detalle de una incidencia
`/asignaciones`	`AsignacionesComponent`	Listado de asignaciones

Arquitectura

La aplicación sigue la arquitectura estándar de Angular con componentes, servicios para las llamadas HTTP a la API REST, e interceptores para inyectar el token JWT en cada petición. El `AdminLayoutComponent` actúa como contenedor principal que renderiza el menú de navegación lateral y el `<router-outlet>` donde se cargan las páginas hijas.

La aplicación utiliza H2 como motor de base de datos embebido, configurado en modo persistente en fichero (`jdbc:h2:data/agroappdb`). Esto permite que los datos persistan entre reinicios de la aplicación sin necesidad de un servidor de base de datos externo, lo que facilita el desarrollo y las pruebas.

Configuración destacada

Propiedad	Valor	Descripción
`ddl-auto`	`create-drop`	Las tablas se crean al arrancar y se eliminan al parar
`server.port`	`9000`	La API escucha en el puerto 9000
`jwt.duration`	`1440 min`	El token de acceso expira en 24 horas

`jwt.refresh.duration`	`10080 min`	El refresh token expira en 7 días
`swagger-ui`	`/swagger-ui.html`	Documentación interactiva de la API
`h2-console`	`/h2-console`	Consola web para inspeccionar la base de datos

Esquema de tablas

El esquema de la base de datos es generado automáticamente por Hibernate a partir de las entidades JPA anotadas. Las tablas principales son:

- `trabajadores` — usuarios del sistema
- `maquinas` — maquinaria agrícola
- `asignaciones` — relación trabajador-máquina con fechas
- `incidencias` — registro de problemas
- `refresh_token` — tokens de refresco de sesión
- `idsequences` — tabla auxiliar para la generación de IDs secuenciales